國立臺北科技大學
National Taipei University of Technology

資料結構
Data Structure

Lab 07

姓名: 盧敬恩
學號: 114310123

Lab07-Ex1. (Linked List Practice)

使用Linked List模擬堆疊,將infix轉換成posfix

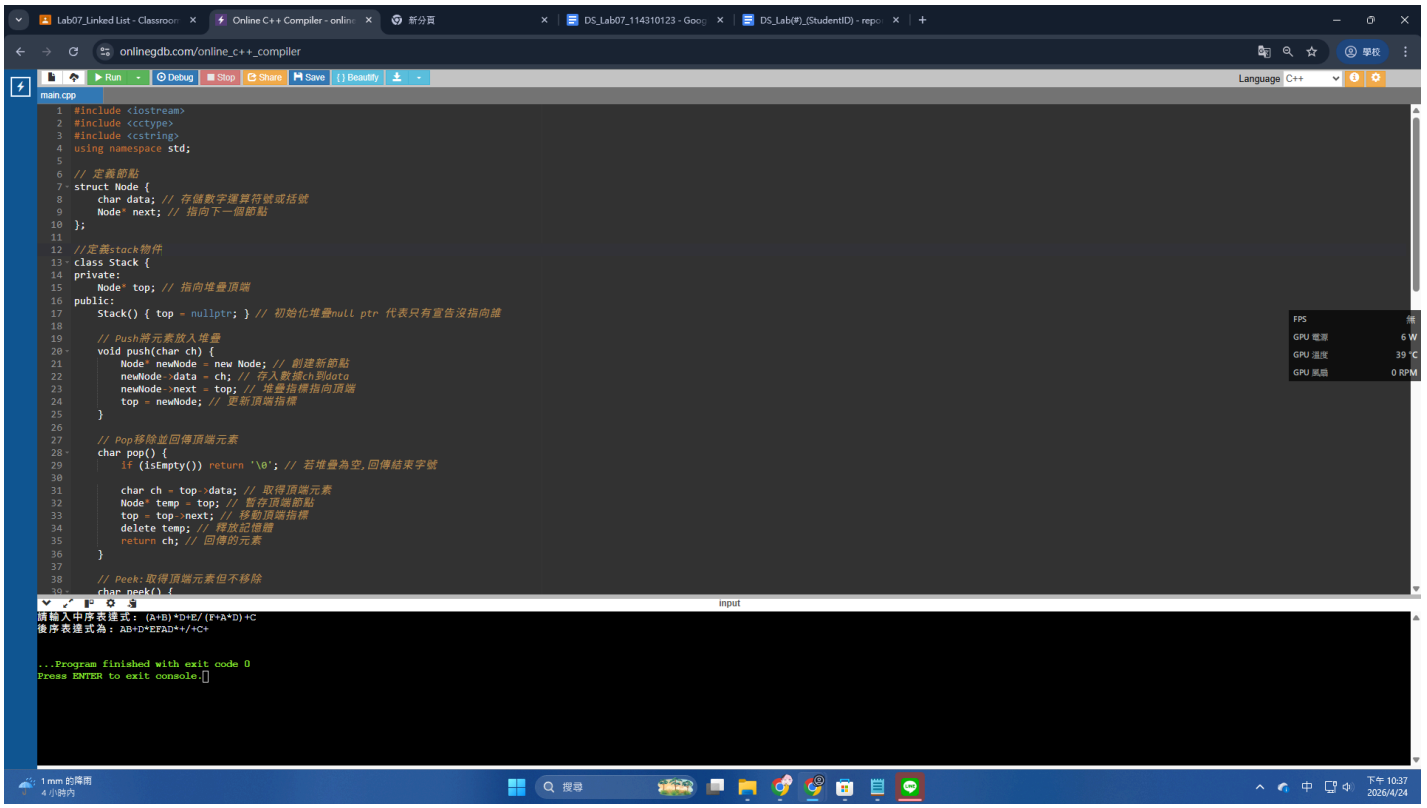
Code

```
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

// 定義節點
struct Node {
        char data; // 存儲數字運算符號或括號
        Node* next; // 指向下一個節點
};

//定義stack物件
class Stack {
private:
        Node* top; // 指向堆疊頂端
public:
        Stack() { top = nullptr; } // 初始化堆疊null ptr 代表只有宣告沒指向誰

        // Push將元素放入堆疊
        void push(char ch) {
                Node* newNode = new Node; // 創建新節點
                newNode->data = ch; // 存入數據ch到data
                newNode->next = top; // 堆疊指標指向頂端
                top = newNode; // 更新頂端指標
        }

        // Pop移除並回傳頂端元素
        char pop() {
                if (isEmpty()) return '\0'; // 若堆疊為空,回傳結束字號

                char ch = top->data; // 取得頂端元素
                Node* temp = top; // 暫存頂端節點
                top = top->next; // 移動頂端指標
                delete temp; // 釋放記憶體
                return ch; // 回傳的元素
        }

        // Peek:取得頂端元素但不移除
        char peek() {
                return (top ? top->data : '\0');
        }

        // 判斷堆疊是否為空
        bool isEmpty() {
                return top == nullptr;
        }




        // 定義運算子優先等級
        int precedence(char op) {
                if (op == '+' || op == '-') return 1;
                if (op == '*' || op == '/') return 2;
                return 0;
        }
```

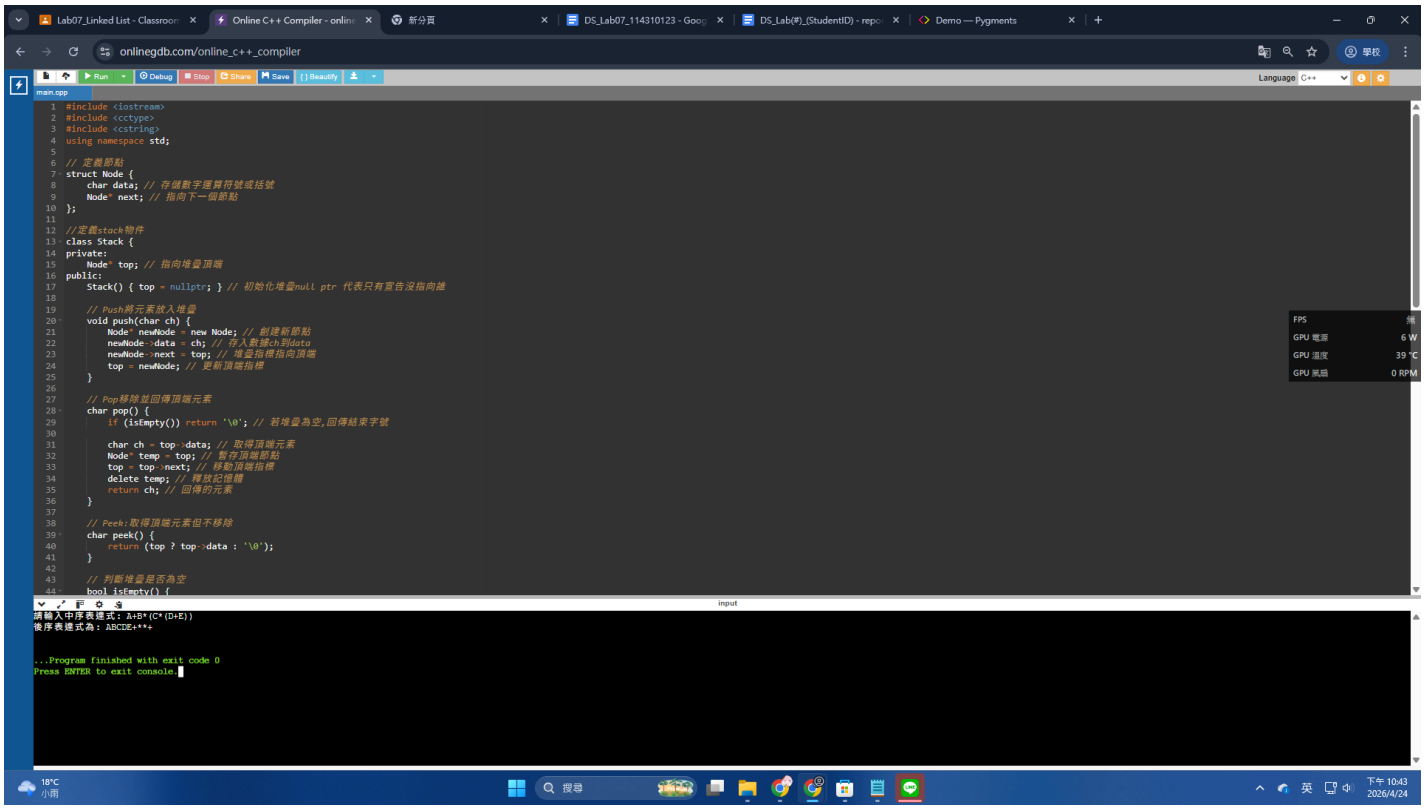
```
        // 將中序轉為後序
        void InfixToPostfix(const char* infix, char* postfix) {//放入整個array
                int i = 0; //  infix
                int j = 0; //  postfix

                while (infix[i] != '\0') {// \0 是停止符號
                        char token = infix[i];
                        // 如果是字母或數字，直接輸出到後序
                        if (isalnum(token)) {//is alnim可以判斷該token是否為數字或是a-z字母
                                postfix[j++] = token;
                        }
                        // 如果是左括號，直接 Push 進堆疊
                        else if (token == '(') {
                                push(token);
                        }
                        // 如果是右括號，Pop 堆疊直到遇見左括號
                        else if (token == ')') {
                                while (!isEmpty() && peek() != '(') {//假設堆疊還有東西 且堆疊指標不為"("就不斷pop
                                        postfix[j++] = pop();//這裡因為pop會return 所以在還沒讀到 (前的所有東西都會輸出
                                }
                                pop(); // 彈出左括號 '(' 但不輸出
                        }
                        // 不是括號也不是字母數字時就是運算子
                        else {
                // 當堆疊不為空，且堆疊最上層的優先權大於等於目前運算子時，彈出該運算子並輸出直到清空或是")"
                                while (!isEmpty() && precedence(peek()) >= precedence(token)) {
                                        postfix[j++] = pop();
                                }
                                push(token);//將當前運算子推入堆疊
                        }
                        i++;
                }
                // 將堆疊中剩餘的運算子全部彈出
                while (!isEmpty()) {
                        postfix[j++] = pop();
                }
                 postfix[j] = '\0'; // 加上字串結束標誌


        }
};

int main() {
        Stack s;
        char infix[100];
        char postfix[100];
        cout << "請輸入中序表達式： ";
        cin >> infix;
        s.InfixToPostfix(infix, postfix);
        cout << "後序表達式為： " << postfix << endl;
        return 0;
}
```

Result

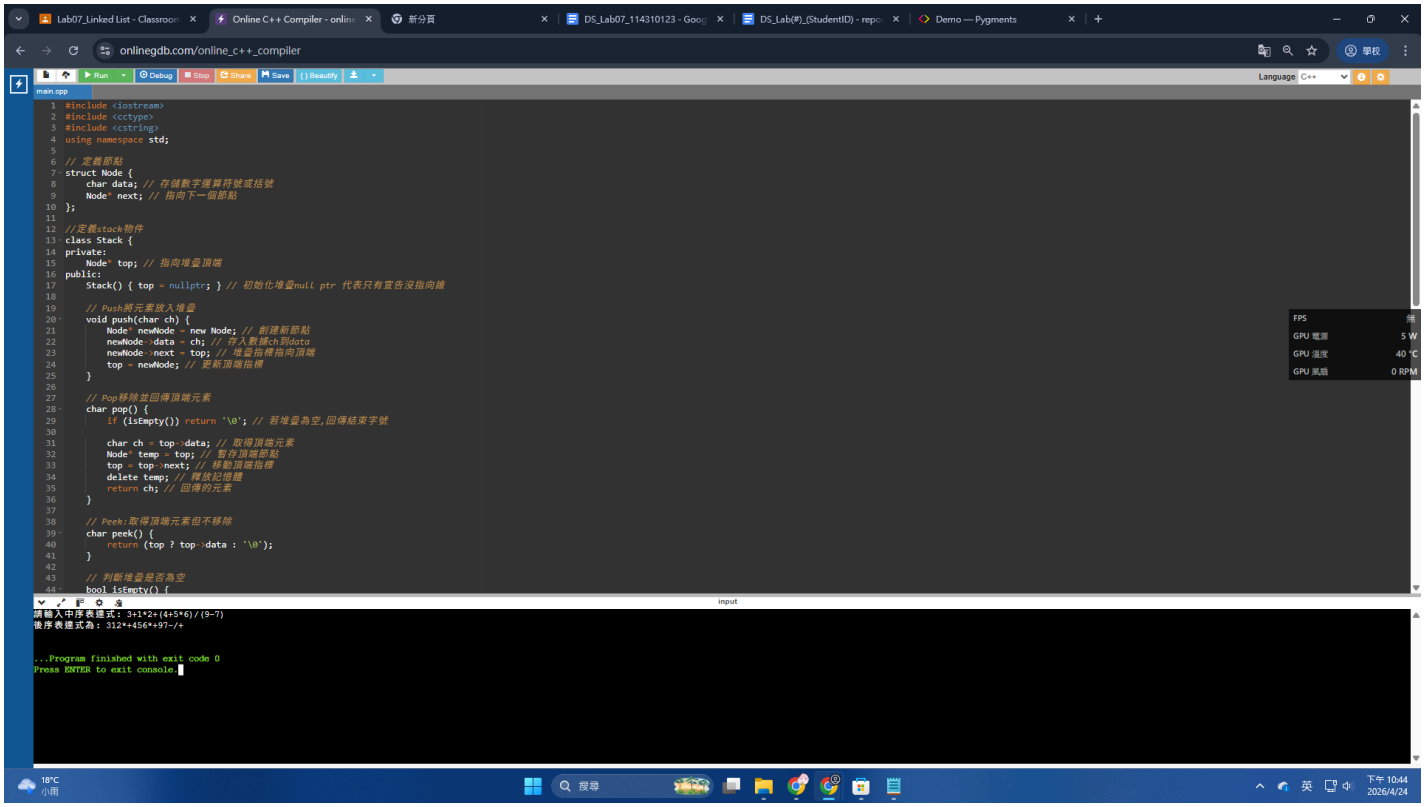
(A+B)*D+E/(F+A*D)+C+ ======> AB+D*EFAD*+/+C+



A+B*(C*(D+E)) =====> ABCDE+**+



3+1*2+(4+5*6)/(9-7) =====>312*+456*+97-/+



畫出每題Infix轉Postfix過程的表格(按照課堂講義作法)

1:(A+B)*D+E/(F+A*D)+C=====> AB+D*EFAD*+/+C+

	A	B	C	D
1	Token(infix)	Stack (0-n)	Top	Output (Postfix)
2	(	(	0	
3	A	(	0	A
4	+	(, +	1	A
5	B	(, +	1	AB
6	)		-1	AB+
7	*	*	0	AB+
8	D	*	0	AB+D
9	+	+	0	AB+D*
10	E	+	0	AB+D*E
11	/	+, /	1	AB+D*E
12	(	+, /, (	2	AB+D*E
13	F	+, /, (	2	AB+D*EF
14	+	+, /, (, +	3	AB+D*EF
15	A	+, /, (, +	3	AB+D*EFA
16	*	+, /, (, +, *	4	AB+D*EFA
17	D	+, /, (, +, *	4	AB+D*EFAD
18	)	+, /	1	AB+D*EFAD*+
19	+	+	0	AB+D*EFAD*+/
20	C	+	0	AB+D*EFAD*+/C
21	<'\0'>		-1	AB+D*EFAD*+/+C+

A+B*(C*(D+E)) =====>ABCDE+**+

	A	B	C	D
1	Token	Stack	Top	Output (Postfix)
2	A		-1	A
3	+	+	0	A
4	B	+	0	AB
5	*	+, *	1	AB
6	(	+, *, (	2	AB
7	C	+, *, (	2	ABC
8	*	+, *, (, *	3	ABC
9	(	+, *, (, *, (	4	ABC
10	D	+, *, (, *, (	4	ABCD
11	+	+, *, (, *, (, +	5	ABCD
12	E	+, *, (, *, (, +	5	ABCDE
13	)	+, *, (, *	3	ABCDE+
14	)	+, *	1	ABCDE+*
15	<eos>		-1	ABCDE+**+

3+1*2+(4+5*6)/(9-7) =====> 312*+456*+97-/+

	A	B	C	D
1	Token	Stack (Content)	Top	Output (Postfix)
2	3		-1	3
3	+	+	0	3
4	1	+	0	31
5	*	+, *	1	31
6	2	+, *	1	312
7	+	+	0	312*+
8	(	+, (	1	312*+
9	4	+, (	1	312*+4
10	+	+, (, +	2	312*+4
11	5	+, (, +	2	312*+45
12	*	+, (, +, *	3	312*+45
13	6	+, (, +, *	3	312*+456
14	)	+	0	312*+456*+
15	/	+, /	1	312*+456*+
16	(	+, /, (	2	312*+456*+
17	9	+, /, (	2	312*+456*+9
18	-	+, /, (, -	3	312*+456*+9
19	7	+, /, (, -	3	312*+456*+97
20	)	+, /	1	312*+456*+97-
21	<eos>		-1	312*+456*+97-/+